Calculating Correct and Efficient Programs in Rocq

David Castro-Perez

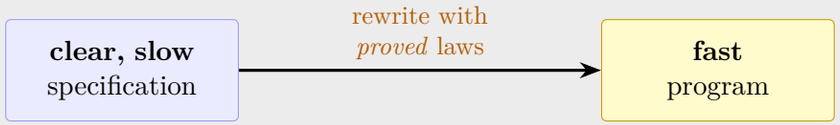
David.CastroPerez@rhul.ac.uk



Based on joint work with Marco Paviotti and Michael Vollmer (University of Kent):
Program Optimisations via Hylomorphisms for Extraction of Executable Code, ITP 2025.

- Joint work with Marco Paviotti and Michael Vollmer (ITP 2025)
- Theme: fast *and* provably correct, without a new proof per program

In One Slide



The idea (1980s):

- Programs as algebra
- Optimise by rewriting
- Correct by construction

Our work:

- Every step machine-checked
- No extra axioms
- Runnable code out

In One Slide

- Point at the arrow: spec in, fast program out
- Each step is an equation, so the result equals the spec
- Old idea (Bird, Meertens); new: every step checked by Rocq, code you can run

The Problem

Fast *and* Correct



A Specification and a Program

- Spec: sort, then double. Two passes, a throwaway list
- Program: doubles each pivot as it goes. One pass
- Equal? Ask the audience. Goal: derive it, not check it

A Specification and a Program




```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

A Specification and a Program

- Spec: sort, then double. Two passes, a throwaway list
- Program: doubles each pivot as it goes. One pass
- Equal? Ask the audience. Goal: derive it, not check it

A Specification and a Program




```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

Are they equal? Can we *derive* the second from the first?

A Specification and a Program

- Spec: sort, then double. Two passes, a throwaway list
- Program: doubles each pivot as it goes. One pass
- Equal? Ask the audience. Goal: derive it, not check it

Three Ways to Fast and Correct Code

Approach	Good	But
Write it, then **verify** it	any program	a new proof for every program

Three Ways to Fast and Correct Code

- Verify afterwards: works for anything, but a new proof each time
- Verified compilers (CompCert, CakeML): faithful, but keep the algorithm
- Calculation: proof is a by-product; needs general laws

Three Ways to Fast and Correct Code

Approach	Good	But
Write it, then **verify** it	any program	a new proof for every program
Verified compiler	translation is correct	keeps your algorithm: $O(n^3)$ stays $O(n^3)$

Three Ways to Fast and Correct Code

- Verify afterwards: works for anything, but a new proof each time
- Verified compilers (CompCert, CakeML): faithful, but keep the algorithm
- Calculation: proof is a by-product; needs general laws

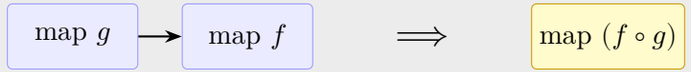
Three Ways to Fast and Correct Code

Approach	Good	But
Write it, then **verify** it	any program	a new proof for every program
Verified compiler	translation is correct	keeps your algorithm: $O(n^3)$ stays $O(n^3)$
Calculate it from a spec	proof comes with the derivation	needs laws that *apply*

Three Ways to Fast and Correct Code

- Verify afterwards: works for anything, but a new proof each time
- Verified compilers (CompCert, CakeML): faithful, but keep the algorithm
- Calculation: proof is a by-product; needs general laws

You Already Trust Rewrite Rules

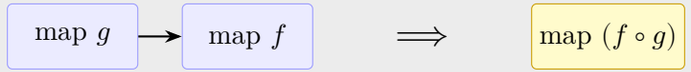


- ML compilers *fuse* tensor operations into one kernel.

You Already Trust Rewrite Rules

- Map fusion: two passes become one
- ML compilers fuse tensor ops (XLA, TVM)
- query optimisers
- compilers: inlining, loop fusion; GHC rewrite rules
- but: fixed rules, trusted; a wrong rule miscompiles silently

You Already Trust Rewrite Rules

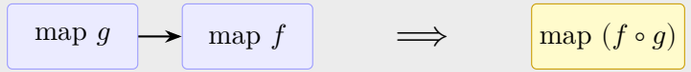


- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.

You Already Trust Rewrite Rules

- Map fusion: two passes become one
- ML compilers fuse tensor ops (XLA, TVM)
- query optimisers
- compilers: inlining, loop fusion; GHC rewrite rules
- but: fixed rules, trusted; a wrong rule miscompiles silently

You Already Trust Rewrite Rules

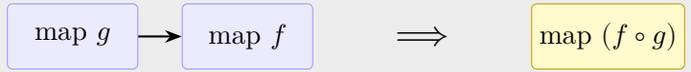


- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.
- Compilers inline, fuse loops, remove temporary data.

You Already Trust Rewrite Rules

- Map fusion: two passes become one
- ML compilers fuse tensor ops (XLA, TVM)
- query optimisers
- compilers: inlining, loop fusion; GHC rewrite rules
- but: fixed rules, trusted; a wrong rule miscompiles silently

You Already Trust Rewrite Rules



- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.
- Compilers inline, fuse loops, remove temporary data.
- Usually: a **fixed** set of rules, **trusted**, not proved.

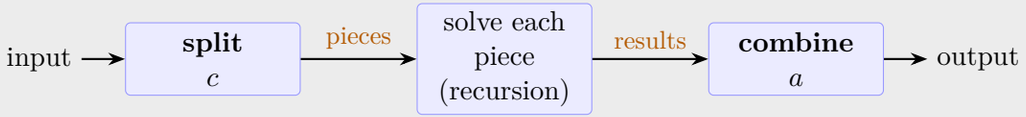
You Already Trust Rewrite Rules

- Map fusion: two passes become one
- ML compilers fuse tensor ops (XLA, TVM)
- query optimisers
- compilers: inlining, loop fusion; GHC rewrite rules
- but: fixed rules, trusted; a wrong rule miscompiles silently

The Laws

Divide and Conquer as Algebra

One Shape for Many Algorithms

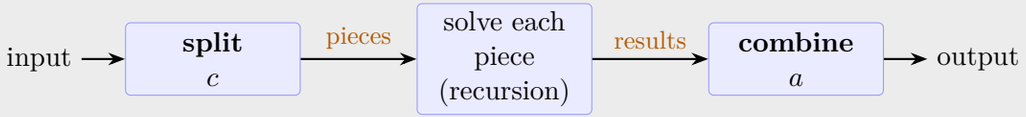


Algorithm	**split** c	**combine** a

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

One Shape for Many Algorithms

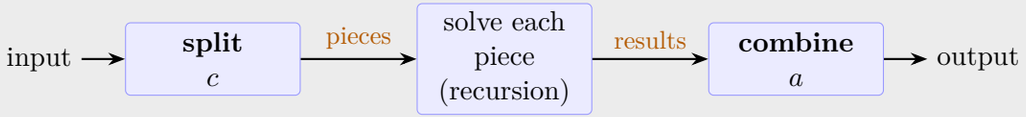


Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

One Shape for Many Algorithms

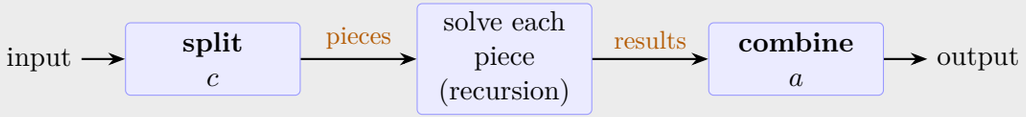


Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

One Shape for Many Algorithms

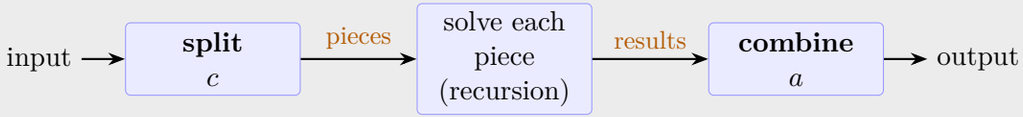


Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge
x^n	$n \mapsto$ last bit, $n/2$	square, maybe multiply by x

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

One Shape for Many Algorithms



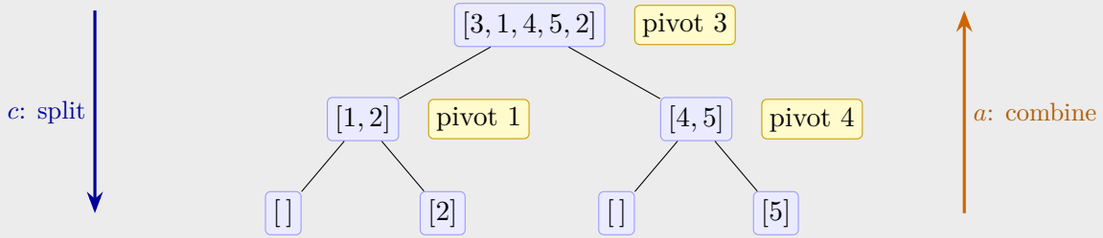
Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge
x^n	$n \mapsto$ last bit, $n/2$	square, maybe multiply by x

Hylomorphism $[\![a, c]\!]$: the recursion is *generic*; only a and c change.

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

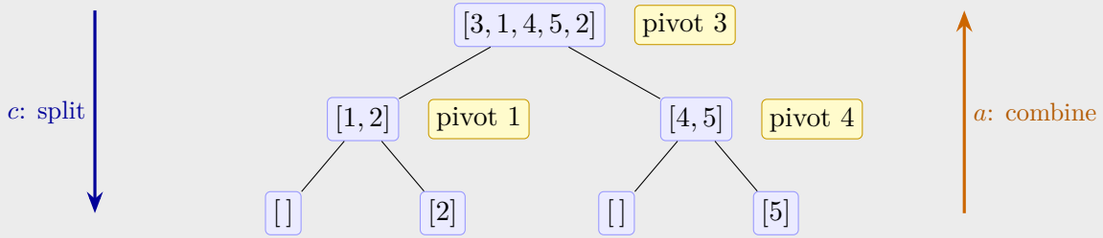
The Call Tree of Quicksort



The Call Tree of Quicksort

- Split goes down, combine comes up
- the tree is only the call stack, never in memory

The Call Tree of Quicksort

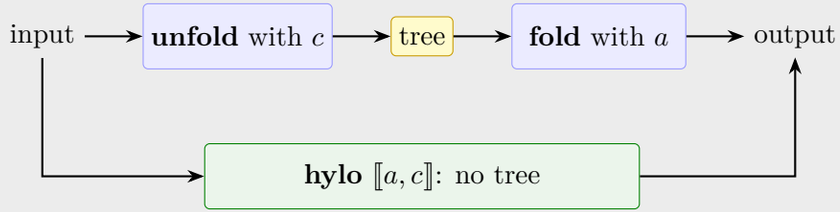


The tree is *virtual*: it is never built in memory.

The Call Tree of Quicksort

- Split goes down, combine comes up
- the tree is only the call stack, never in memory

Producers and Consumers

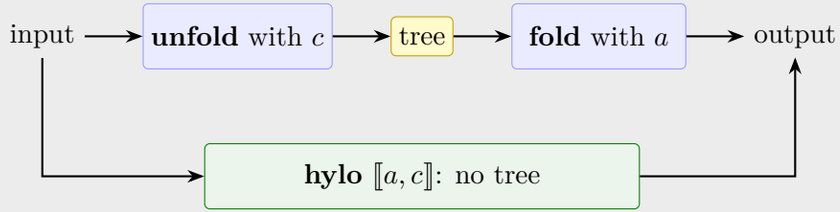


- Specification: *build*, then *consume*. Easy to read.

Producers and Consumers

- Top: build then consume; easy to read, wastes memory
- Bottom: hylo, no tree (deforestation, Wadler 1990)
- specs look like the top, programs like the bottom

Producers and Consumers



- Specification: *build*, then *consume*. Easy to read.
- Program: do both at once. Fast.

Producers and Consumers

- Top: build then consume; easy to read, wastes memory
- Bottom: hylo, no tree (deforestation, Wadler 1990)
- specs look like the top, programs like the bottom

Two Laws

Fusion $[\![a, c]\!] \rightarrow h = [\![b, c]\!]$

if h distributes over a

Deforestation $[\![\mathsf{build}, c]\!] \rightarrow [\![a, \mathsf{take_apart}]\!] = [\![a, c]\!]$

- Proved **once**, for every split, combine, data type.

Two Laws

- Fusion: push post-processing into combine, if it distributes
- Deforestation: build then take apart cancels out
- proved once, for every data type
- an optimisation is a chain of these

Two Laws

Fusion $[\![a, c]\!] \rightarrow h = [\![b, c]\!]$

if h distributes over a

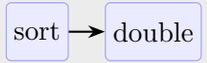
Deforestation $[\![\mathsf{build}, c]\!] \rightarrow [\![a, \mathsf{take_apart}]\!] = [\![a, c]\!]$

- Proved **once**, for every split, combine, data type.
- Each use is a *rewrite step*.

Two Laws

- Fusion: push post-processing into combine, if it distributes
- Deforestation: build then take apart cancels out
- proved once, for every data type
- an optimisation is a chain of these

Deriving the Example



Deriving the Example

- Quicksort = hylo with partition and concat
- doubling distributes over concat: fusion
- result: the hand-written program
- each step a theorem, applied in Rocq

Deriving the Example

$$\mathrm{sort} \to \mathrm{double}$$

$$= \quad [\![\mathrm{concat}, \mathrm{partition}]\!] \to \mathrm{double} \qquad \text{definition of quicksort}$$

Deriving the Example

- Quicksort = hylo with partition and concat
- doubling distributes over concat: fusion
- result: the hand-written program
- each step a theorem, applied in Rocq

Deriving the Example

$$\text{sort} \to \text{double}$$

$$= \quad [\![\text{concat}, \text{partition}]\!] \to \text{double} \qquad \text{definition of quicksort}$$

$$= \quad [\![\text{concat and double pivot}, \text{partition}]\!] \qquad \text{fusion}$$

Deriving the Example

- Quicksort = hylo with partition and concat
- doubling distributes over concat: fusion
- result: the hand-written program
- each step a theorem, applied in Rocq

Deriving the Example

$$\text{sort} \to \text{double}$$

$$= \quad [\![\text{concat}, \text{partition}]\!] \to \text{double} \qquad \text{definition of quicksort}$$

$$= \quad [\![\text{concat and double pivot}, \text{partition}]\!] \qquad \text{fusion}$$

Each $=$ is a **theorem** applied by a proof assistant.

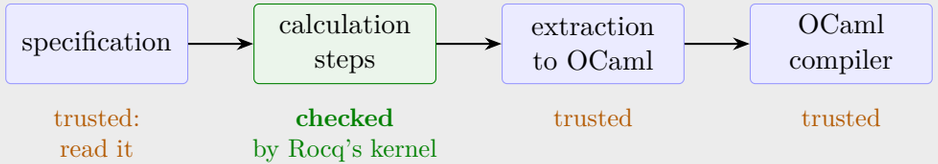
Deriving the Example

- Quicksort = hylo with partition and concat
- doubling distributes over concat: fusion
- result: the hand-written program
- each step a theorem, applied in Rocq

Machine-Checked

Hylomorphisms in Rocq

What You Have to Trust

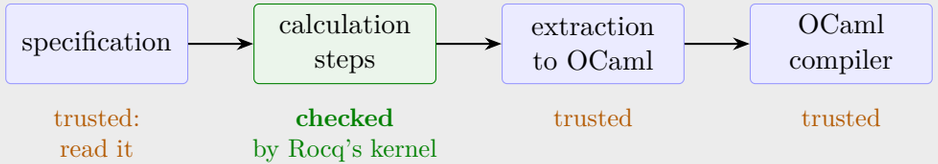


- Who picked the steps does not matter: a person, a script, a model.

What You Have to Trust

- Green box: checked by Rocq's small kernel
- Trusted: spec, kernel, extraction, OCaml compiler
- steps can come from anyone: person, script, model
- wrong step = rejected

What You Have to Trust



- Who picked the steps does not matter: a person, a script, a model.
- A wrong step is **rejected**, not miscompiled.

What You Have to Trust

- Green box: checked by Rocq's small kernel
- Trusted: spec, kernel, extraction, OCaml compiler
- steps can come from anyone: person, script, model
- wrong step = rejected

Three Obstacles

Termination
every function must provably stop

No extra axioms
comparing functions needs one

Clean code
generic proofs extract to unsafe casts

Three Obstacles

- Termination: Rocq only accepts recursion that provably stops
- Axioms: function equality needs one; we avoid it
- Clean code: textbook encoding extracts badly (next slide)
- Haskell has the laws, no termination

Three Obstacles

Termination
every function must provably stop

No extra axioms
comparing functions needs one

Clean code
generic proofs extract to unsafe casts

Haskell has the laws but not termination; Rocq has termination but rejects the textbook definitions.

Three Obstacles

- Termination: Rocq only accepts recursion that provably stops
- Axioms: function equality needs one; we avoid it
- Clean code: textbook encoding extracts badly (next slide)
- Haskell has the laws, no termination

What the Textbook Encoding Extracts To

```
type sh = __                                    (* = Obj.t *)
type pos = __
type 'x ext = { shape : sh; cont : (pos -> 'x) }

let merge x =
  let { shape = shape0; cont = k } = x in
  (match Obj.magic shape0 with
   | Some h -> app (Obj.magic k True) (Cons (h, (Obj.magic k False)))
   | None -> Nil)
```

(simplified)

What the Textbook Encoding Extracts To

- Real extraction output, trimmed
- point at `Obj.magic`: unchecked casts everywhere

What the Textbook Encoding Extracts To

```
type sh = __                                    (* = Obj.t *)
type pos = __
type 'x ext = { shape : sh; cont : (pos -> 'x) }

let merge x =
  let { shape = shape0; cont = k } = x in
  (match Obj.magic shape0 with
   | Some h -> app (Obj.magic k True) (Cons (h, (Obj.magic k False)))
   | None -> Nil)
```

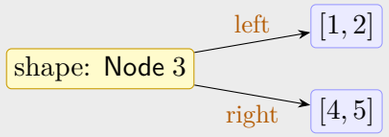
(simplified)

`Obj.magic`: an **unchecked cast**. Unsafe as soon as this code meets other code.

What the Textbook Encoding Extracts To

- Real extraction output, trimmed
- point at `Obj.magic`: unchecked casts everywhere

Our Fix: Shapes and Positions



shape: Leaf no positions

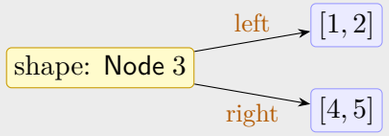
valid?	left	right
`Node h`	yes	yes
`Leaf`	no	no

- One type of positions + a **yes/no check**: plain OCaml types.

Our Fix: Shapes and Positions

- Data = shape + positions (containers)
- Point at the table: one position type, yes/no validity
- plain OCaml types
- proofs erased at extraction: no casts

Our Fix: Shapes and Positions



shape: Leaf no positions

valid?	left	right
`Node h`	yes	yes
`Leaf`	no	no

- One type of positions + a **yes/no check**: plain OCaml types.
- Proofs about the check are **erased** from the code.

Our Fix: Shapes and Positions

- Data = shape + positions (containers)
- Point at the table: one position type, yes/no validity
- plain OCaml types
- proofs erased at extraction: no casts

Termination: Recursion on a Proof



- Proved **once per split**, e.g. “pieces are shorter”.

Termination: Recursion on a Proof

- Proof tree has the same shape as the call tree; recurse on it
- one proof per split, e.g. pieces are shorter
- proof erased: code is just the recursion

Termination: Recursion on a Proof



- Proved **once per split**, e.g. “pieces are shorter”.
- The proof is scaffolding: **erased** at extraction.

Termination: Recursion on a Proof

- Proof tree has the same shape as the call tree; recurse on it
- one proof per split, e.g. pieces are shorter
- proof erased: code is just the recursion

No Extra Axioms

```
Print Assumptions hylo_fusion_l.          Closed under the global context
Print Assumptions mss_calc.               Closed under the global context
Print Assumptions qsort_times_two_calc.   Axioms: PrimInt63.int, ltb, mul
```

(simplified)

- Equality = *same results on equal inputs* (setoids).

No Extra Axioms

- Point at Closed under the global context
- Only Rocq's machine integers otherwise
- equality = same results on equal inputs
- laws usable as rewrite rules

No Extra Axioms

```
Print Assumptions hylo_fusion_l.          Closed under the global context
Print Assumptions mss_calc.               Closed under the global context
Print Assumptions qsort_times_two_calc.   Axioms: PrimInt63.int, ltb, mul
```

(simplified)

- Equality = *same results on equal inputs* (setoids).
- Laws become `rewrite` steps.

No Extra Axioms

- Point at Closed under the global context
- Only Rocq's machine integers otherwise
- equality = same results on equal inputs
- laws usable as rewrite rules

In Practice

From Calculation to Code

A Calculation in Rocq

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  finished.
Qed.
```

A Calculation in Rocq

- `Derive`: the program starts as an unknown
- the spec: quicksort, then double
- one line of laws; Rocq fills in the program

A Calculation in Rocq

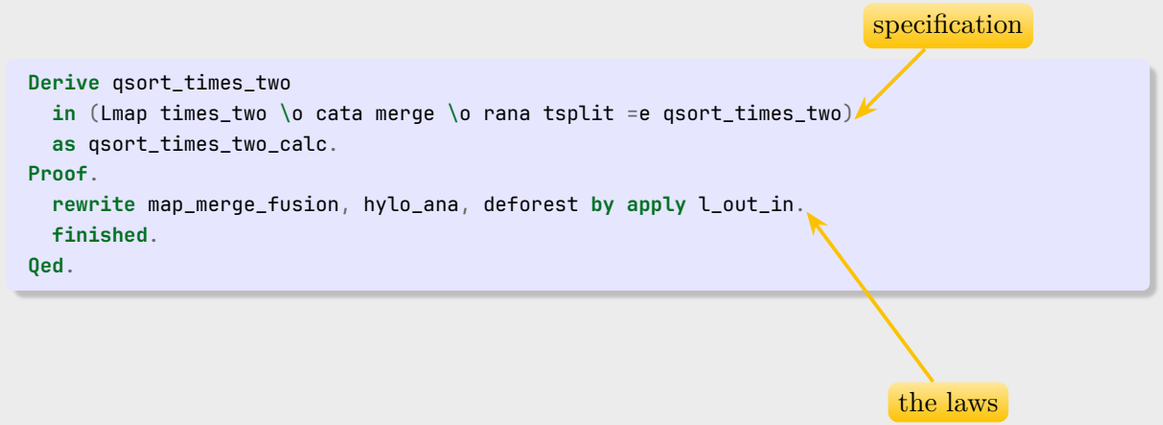
specification

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  finished.
Qed.
```

A Calculation in Rocq

- `Derive`: the program starts as an unknown
- the spec: quicksort, then double
- one line of laws; Rocq fills in the program

A Calculation in Rocq



A Calculation in Rocq

- `Derive`: the program starts as an unknown
- the spec: quicksort, then double
- one line of laws; Rocq fills in the program

What Comes Out

```
let rec qsort_times_two_code = function
| [] -> []
| h :: t ->
  let (l, r) = partition (fun x0 -> ltb x0 h) t in
  let x0 = fun e ->
    qsort_times_two_code (match e with Lbranch -> l | Rbranch -> r) in
  app (x0 Lbranch) ((mul (Uint63.of_int (2)) h) :: (x0 Rbranch))
```

(simplified)

What Comes Out

- Extracted: one pass, doubles each pivot
- compare: same as the hand-written one, now with a proof

What Comes Out

```
let rec qsort_times_two_code = function
| [] -> []
| h :: t ->
  let (l, r) = partition (fun x0 -> ltb x0 h) t in
  let x0 = fun e ->
    qsort_times_two_code (match e with Lbranch -> l | Rbranch -> r) in
  app (x0 Lbranch) ((mul (Uint63.of_int (2)) h) :: (x0 Rbranch))
```

(simplified)

Hand-written (start of the talk):

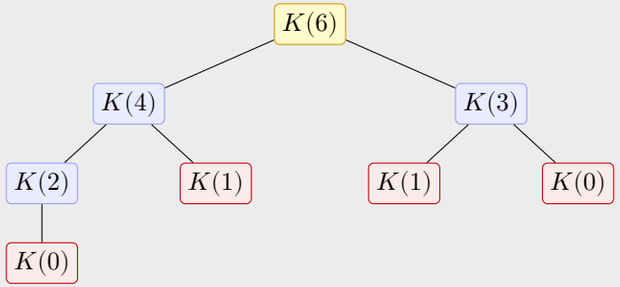
```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

What Comes Out

- Extracted: one pass, doubles each pivot
- compare: same as the hand-written one, now with a proof

Dynamic Programming: Knapsack

$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \leq c \,\}\right)$$

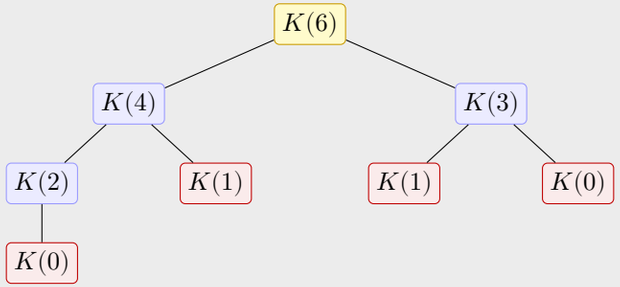


Dynamic Programming: Knapsack

- Classic DP recurrence
- Point at red nodes: the same capacities computed again and again
- naive recursion is a hylo, but exponential

Dynamic Programming: Knapsack

$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \leq c \,\}\right)$$

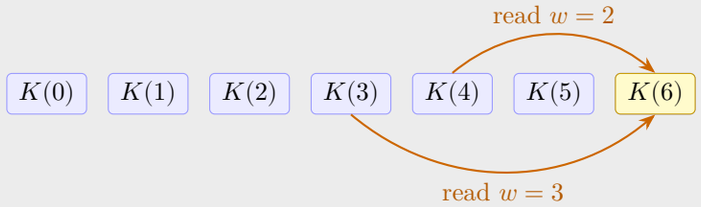


A hylomorphism, but it **recomputes**: exponential.

Dynamic Programming: Knapsack

- Classic DP recurrence
- Point at red nodes: the same capacities computed again and again
- naive recursion is a hylo, but exponential

Remember, Don't Recompute



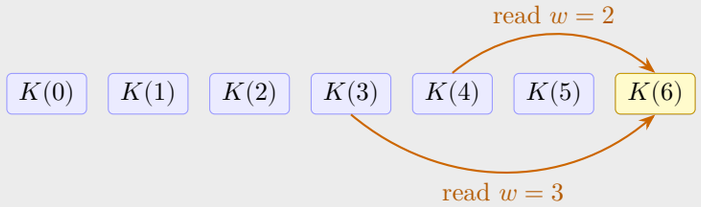
- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

Remember, Don't Recompute

- Keep a table of results for every capacity
- Point at arrows: K(6) reads K(4) and K(3) from the table
- dynamorphism (Hinze, Wu, Gibbons)
- law (`reenumerate`): change what we recurse on, items to capacities
- exponential to one entry per capacity

Remember, Don't Recompute



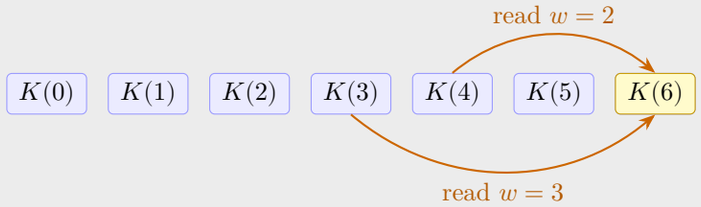
- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.
- A law: recursion over *items* = recursion over *capacities* with a table, if each step *looks up* its subproblems.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

Remember, Don't Recompute

- Keep a table of results for every capacity
- Point at arrows: K(6) reads K(4) and K(3) from the table
- dynamorphism (Hinze, Wu, Gibbons)
- law (`reenumerate`): change what we recurse on, items to capacities
- exponential to one entry per capacity

Remember, Don't Recompute



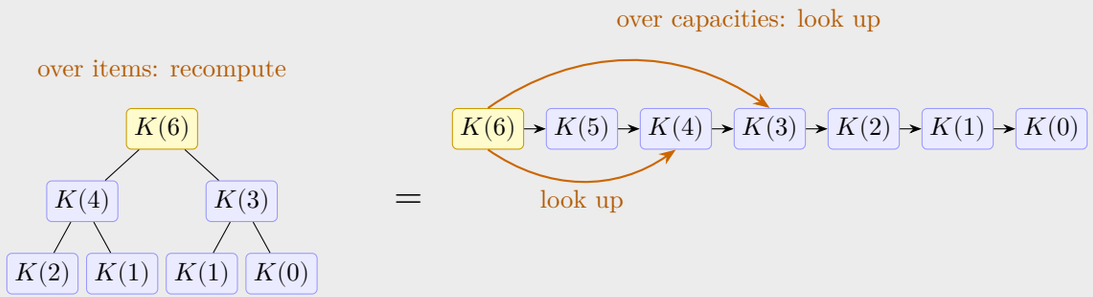
- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.
- A law: recursion over *items* = recursion over *capacities* with a table, if each step *looks up* its subproblems.
- Exponential $\Longrightarrow$ one entry per capacity.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

Remember, Don't Recompute

- Keep a table of results for every capacity
- Point at arrows: K(6) reads K(4) and K(3) from the table
- dynamorphism (Hinze, Wu, Gibbons)
- law (`reenumerate`): change what we recurse on, items to capacities
- exponential to one entry per capacity

From Recomputation to a Table, by Rewriting



From Recomputation to a Table, by Rewriting

- Left: naive recursion over items; repeats
- Right: count down capacities; each entry looks up earlier ones
- step 1: a Derive makes the algebra read through a lookup
- step 2: `reenumerate` switches the recursion; tables are persistent arrays
- `dyna_memo`: share the table; no hand-written index arithmetic

From Recomputation to a Table, by Rewriting

```
Definition knapsack_naive wvs := hylo naive_alg (naive_c wvs).  (* exponential *)

(* Step 1: the algebra reads its subproblems through a lookup *)
Derive knap_alg_ in (forall g b, naive_alg (fmap g (naive_c wvs b))
                               = knap_alg_ (look_rec out_int_c g b))
  as knap_look.

(* Step 2: recurse on capacities instead, tables in persistent arrays *)
Derive knapsack_arr in (knapsack_naive wvs =e knapsack_arr) as knapsack_arr_calc.
Proof.
  unfold knapsack_naive.
  rewrite (reenumerate (U := ATab int) (alg := knap_alg) knap_look).
  rewrite dyna_memo.
  finished.
Qed.
```

(simplified)

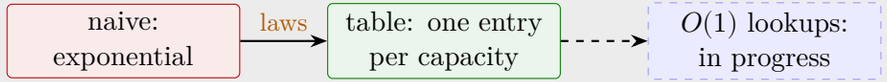
From Recomputation to a Table, by Rewriting

- Left: naive recursion over items; repeats
- Right: count down capacities; each entry looks up earlier ones
- step 1: a Derive makes the algebra read through a lookup
- step 2: `reenumerate` switches the recursion; tables are persistent arrays
- `dyna_memo`: share the table; no hand-written index arithmetic

Knapsack: the Extracted Program

```
let rec look t j =                          (* j steps down the table *)
  if j = 0 then Some (head t)
  else if is_last t then None else look (rest t) (j - 1)
let best wvs t =                            (* t: results for c-1, c-2, ... *)
  max_int 0 (map_some (fun (w, v) ->
    if w = 0 then None else Option.map (fun u -> u + v) (look t (w - 1))) wvs)
let rec table wvs c =
  if c = 0 then single 0
  else let t = table wvs (c - 1) in push (best wvs t) t
let knapsack wvs c = head (table wvs c)
```

(simplified)



Knapsack: the Extracted Program

- `table`: one entry per capacity, built bottom-up
- `look`: walks w steps down; the array table is persistent
- Ladder: exponential to one entry per capacity, all by laws
- Next: index the array directly for O(1) lookups (in progress)

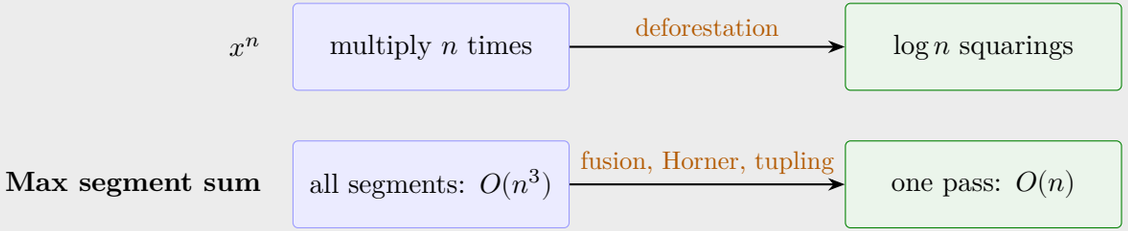
More Calculations



More Calculations

- Fast power: deforest the list of binary digits
- max segment sum: Bird's classic, cubic to linear

More Calculations



More Calculations

- Fast power: deforest the list of binary digits
- max segment sum: Bird's classic, cubic to linear

Maximum Segment Sum: Spec and Result

$$\text{mss} = \text{maximum} \circ \text{map sum} \circ \text{segments}$$

3	−5	2	4	−1

best segment: $2 + 4 = 6$

Maximum Segment Sum: Spec and Result

- Spec is the definition: all segments, sum, max
- one pass, two accumulators; not obviously correct, but proved

Maximum Segment Sum: Spec and Result

$$\text{mss} = \text{maximum} \circ \text{map sum} \circ \text{segments}$$

3	−5	2	4	−1

best segment: $2 + 4 = 6$

```
let mss_code xs =
  let rec f = function
  | [] -> (Some 0, Some 0)
  | h :: t ->
    let (m, s) = f t in
    let s' = match s with
      | Some b -> Some (max 0 (h + b))
      | None -> Some 0 in
    (omax s' m, s')
  in fst (f xs)
```

(simplified)

Maximum Segment Sum: Spec and Result

- Spec is the definition: all segments, sum, max
- one pass, two accumulators; not obviously correct, but proved

Summary So Far

- A **library**: hylomorphisms and their laws in Rocq.
- Optimisations are **proofs by rewriting**.
- Termination proved **once per split**.
- **No extra axioms**; OCaml **without unsafe casts**.

D. Castro-Perez, M. Paviotti, M. Vollmer. **Program Optimisations via Hylomorphisms for Extraction of Executable Code**. ITP 2025. Library: `https://github.com/dcastrop/rocq-hylomorphisms`

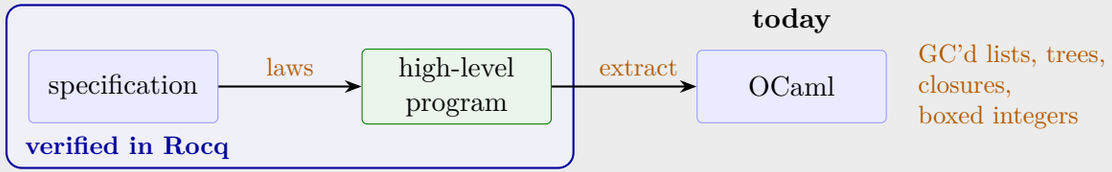
Summary So Far

- The library today: ITP 2025 plus later work
- Next: work in progress

Work in Progress

Down to Low-Level Code

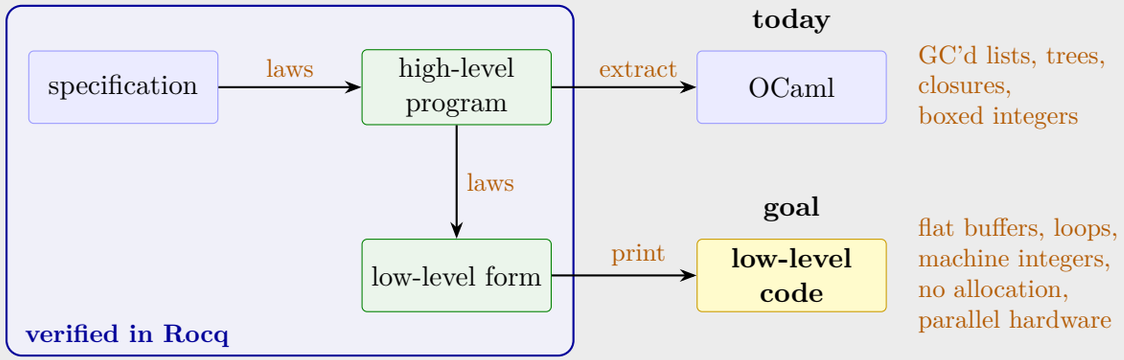
Two Ways Out of Rocq



Two Ways Out of Rocq

- Box = verified in Rocq
- Today: exit through OCaml (GC, closures, boxed integers)
- Goal: more laws to a low-level form, printed directly
- layout and scheduling become calculation steps, not a compiler pass

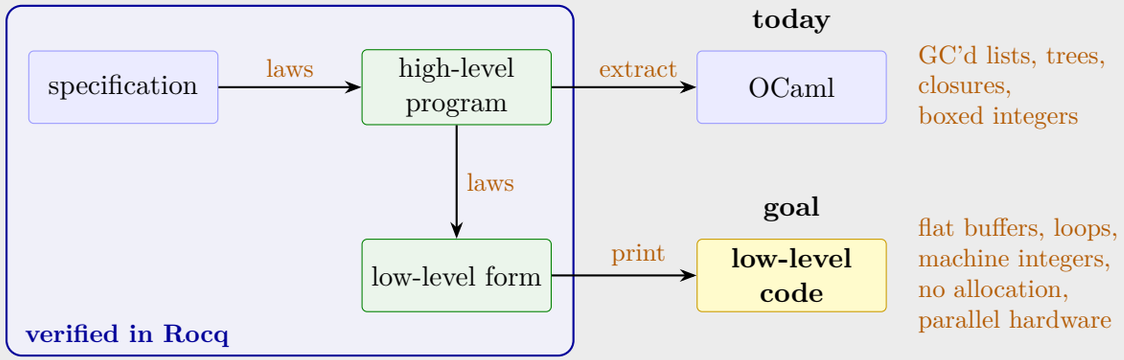
Two Ways Out of Rocq



Two Ways Out of Rocq

- Box = verified in Rocq
- Today: exit through OCaml (GC, closures, boxed integers)
- Goal: more laws to a low-level form, printed directly
- layout and scheduling become calculation steps, not a compiler pass

Two Ways Out of Rocq



Not "OCaml, then lower": **more calculation** reaches the low-level form; extraction is a direct print.

Two Ways Out of Rocq

- Box = verified in Rocq
- Today: exit through OCaml (GC, closures, boxed integers)
- Goal: more laws to a low-level form, printed directly
- layout and scheduling become calculation steps, not a compiler pass

What We Are Working On

Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.

What We Are Working On

- Read the goal box
- you choose laws, not indices
- every step proved
- cost model picks among equal programs

What We Are Working On

Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.
- Every step proved; the output is machine-level code.

What We Are Working On

- Read the goal box
- you choose laws, not indices
- every step proved
- cost model picks among equal programs

What We Are Working On

Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.
- Every step proved; the output is machine-level code.
- A cost model says *which* equal program to pick.

What We Are Working On

- Read the goal box
- you choose laws, not indices
- every step proved
- cost model picks among equal programs

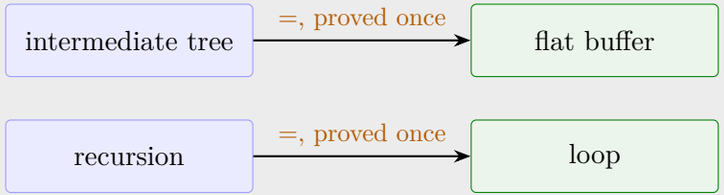
Ongoing: Laws That Change Representation



Ongoing: Laws That Change Representation

- Today's laws only know trees
- recursion to loops
- independent pieces run in parallel
- all proved once, used as rewrites

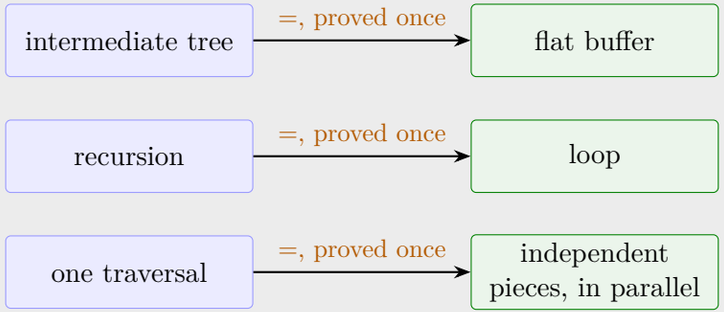
Ongoing: Laws That Change Representation



Ongoing: Laws That Change Representation

- Today's laws only know trees
- recursion to loops
- independent pieces run in parallel
- all proved once, used as rewrites

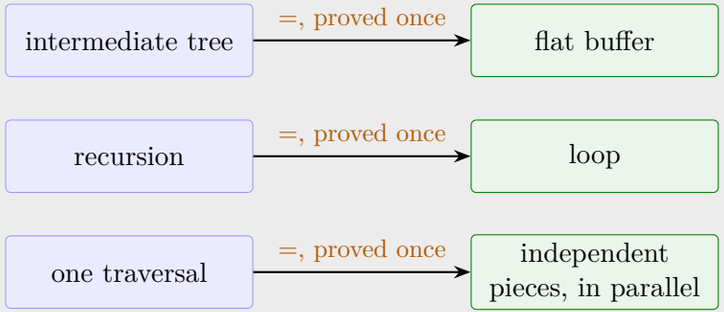
Ongoing: Laws That Change Representation



Ongoing: Laws That Change Representation

- Today's laws only know trees
- recursion to loops
- independent pieces run in parallel
- all proved once, used as rewrites

Ongoing: Laws That Change Representation

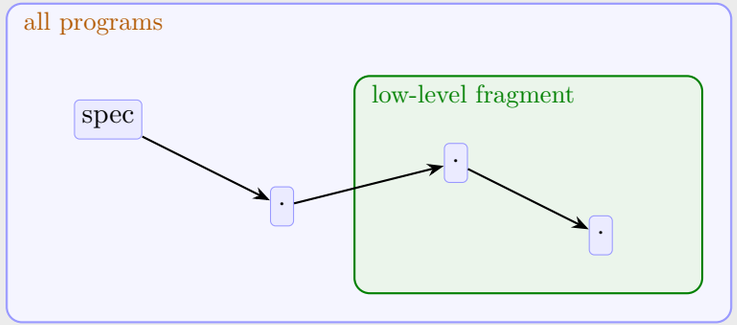


Today's laws change the *algorithm*; these change *where data lives* and *how work is scheduled*.

Ongoing: Laws That Change Representation

- Today's laws only know trees
- recursion to loops
- independent pieces run in parallel
- all proved once, used as rewrites

Ongoing: A Target the Laws Never Leave

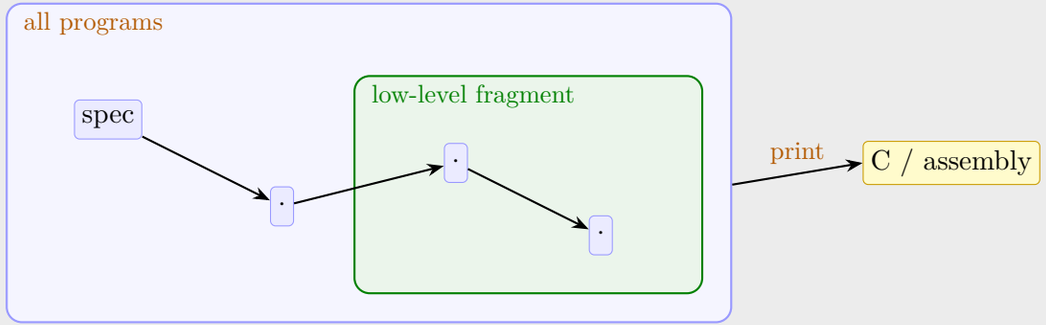


- Every law keeps programs **inside** the fragment.

Ongoing: A Target the Laws Never Leave

- Low-level fragment: no closures, explicit memory, machine integers
- Laws keep programs inside it
- printing is almost syntactic: small trusted base
- overflow and bounds as side conditions

Ongoing: A Target the Laws Never Leave

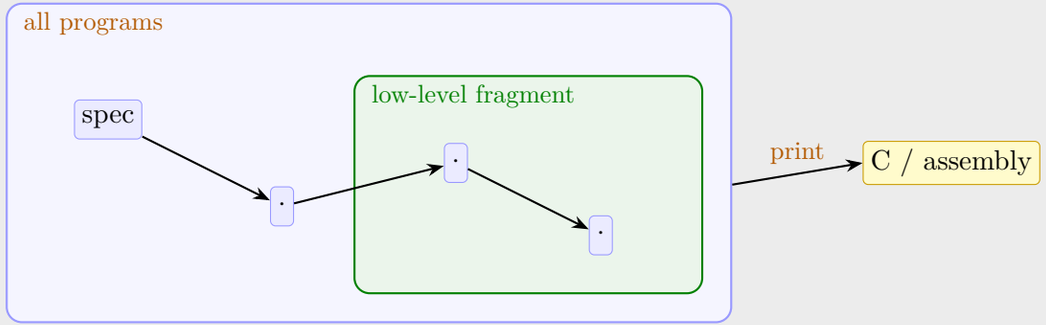


- Every law keeps programs **inside** the fragment.
- Only a **small printer** is trusted.

Ongoing: A Target the Laws Never Leave

- Low-level fragment: no closures, explicit memory, machine integers
- Laws keep programs inside it
- printing is almost syntactic: small trusted base
- overflow and bounds as side conditions

Ongoing: A Target the Laws Never Leave

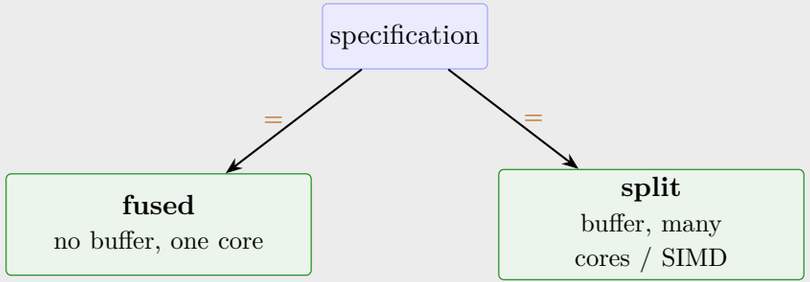


- Every law keeps programs **inside** the fragment.
- Only a **small printer** is trusted.
- Overflow and bounds: **side conditions** of the laws.

Ongoing: A Target the Laws Never Leave

- Low-level fragment: no closures, explicit memory, machine integers
- Laws keep programs inside it
- printing is almost syntactic: small trusted base
- overflow and bounds as side conditions

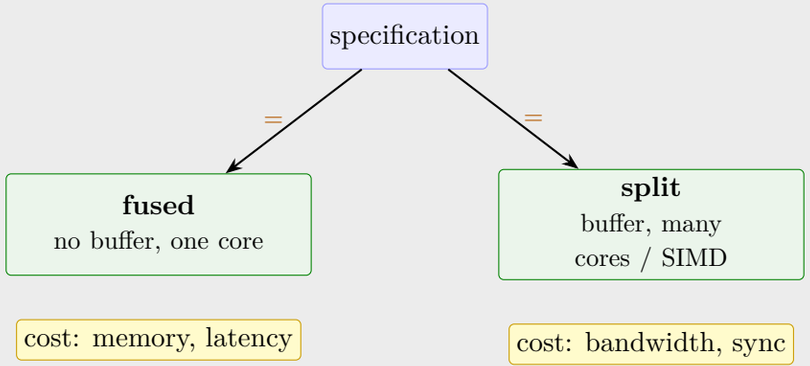
Ongoing: A Cost Model



Ongoing: A Cost Model

- Two equal programs from one spec
- fused saves memory; split enables parallelism
- laws say equal, cost model says better

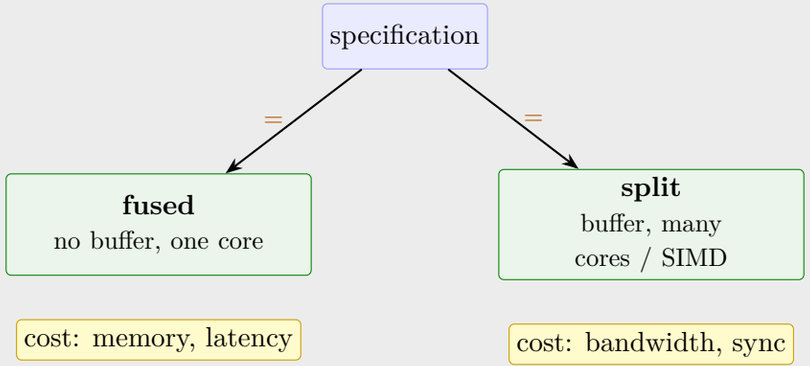
Ongoing: A Cost Model



Ongoing: A Cost Model

- Two equal programs from one spec
- fused saves memory; split enables parallelism
- laws say equal, cost model says better

Ongoing: A Cost Model

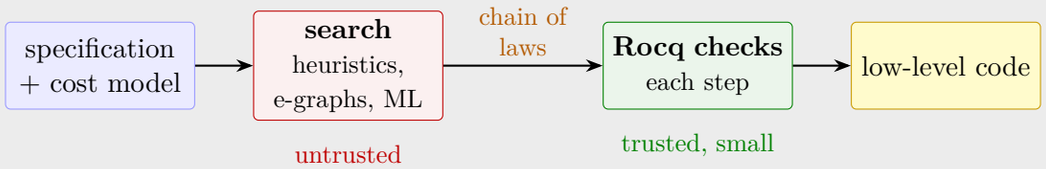


The laws say what is *equal*; a cost model says what is *better*.

Ongoing: A Cost Model

- Two equal programs from one spec
- fused saves memory; split enables parallelism
- laws say equal, cost model says better

Untrusted Search, Trusted Checking

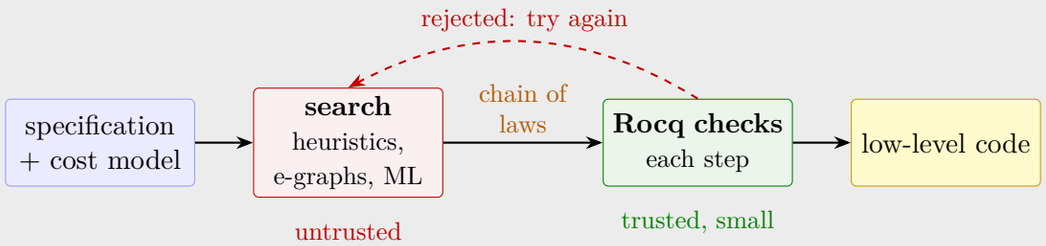


- The derivation is a **certificate**.

Untrusted Search, Trusted Checking

- Search: heuristics, e-graphs, or an ML model
- Rocq checks the chain of laws
- wrong proposals rejected; output correct regardless
- ML: like a model proposing proofs; security: small trusted base

Untrusted Search, Trusted Checking



- The derivation is a **certificate**.
- Search can be as clever, or as wrong, as it likes.

Untrusted Search, Trusted Checking

- Search: heuristics, e-graphs, or an ML model
- Rocq checks the chain of laws
- wrong proposals rejected; output correct regardless
- ML: like a model proposing proofs; security: small trusted base

Where We Are



In Rocq, axiom-free:

- Splitting index ranges k ways
- Machine integers, no overflow (proved)
- Buffers, compared by their reads
- Tree reductions over ranges
- Writes to a buffer, driven by any split

Not yet:

- Laws that add or remove a buffer
- Each index written exactly once
- The low-level target and printer
- The cost model and search

Where We Are

- Early stage; be honest
- Done: exact-cover splits, machine integers, buffers, tree reductions, writes
- Not yet: buffer laws, exactly-once, target, cost model

Summary

- Program optimisation as **algebra**: rewrite with laws.
- Divide and conquer as **hylomorphisms**: laws proved once.
- In Rocq: **machine-checked**, no extra axioms, clean OCaml.
- Ongoing: **low-level, correct-by-construction code** from high-level steps, guided by a **cost model**.

Summary

- Recap the four bullets

Thank you!

Questions?

`https://github.com/dcastrop/rocq-hylomorphisms`

Based on joint work with Marco Paviotti and Michael Vollmer.

- Thank Marco and Michael; questions